

The Badge That Rewrites Itself

A 4.7" e-paper conference badge you edit from your phone, mid-conference, with no laptop, no cable and no reflash — and that keeps its image for months with the power switched off.

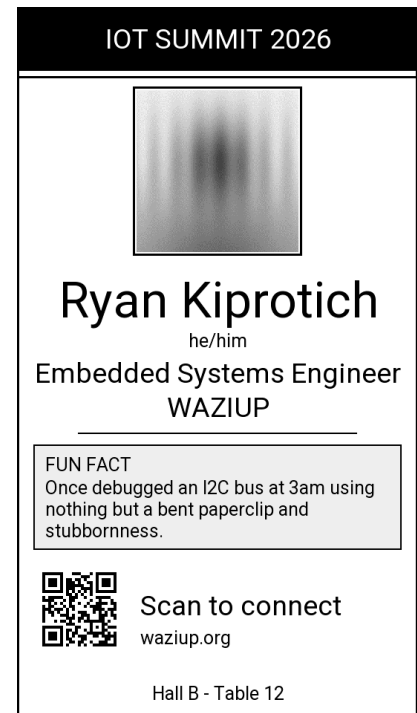
Platform LilyGo T5 4.7" ESP32-S3 (ED047TC1)

Framework Arduino / PlatformIO

Language C++ firmware, vanilla JS setup page, Python preview tool

Build time One evening if the board is already in your hand

Licence GPL-3.0 (inherited from the display driver)



The story: 4,000 people wearing the same dead rectangle

Every conference hands you the same object. A laminated rectangle, your name in whatever the print shop had loaded, your company underneath in a smaller size, and a lanyard that flips the whole thing backwards about ten minutes after you put it on.

Here is the strange part. That badge is the single most-looked-at object at the entire event. In three days it gets read by more people than your website will see in a month. Strangers glance at it for about a second and decide, on that basis alone, whether to start a conversation with you. And it is the one object nobody designs, nobody updates, and nobody can change once it is printed.

I noticed this properly at an IoT event where my badge was wrong by the second morning. It had been printed a week early with an old job title. I was demoing something on day two that was not on it anywhere. I met maybe forty people and every single contact exchange was the same small ritual: unlock phone, open an app, spell out an email address, get it wrong, spell it again. Meanwhile the thing hanging around my neck — the thing everyone was already looking at — sat there saying nothing useful.

So the badge became a project. Not a novelty blinky badge, and not a tiny screen on a lanyard. Something that still reads like a properly printed badge across a crowded room, but which I can rewrite from my phone while standing in the coffee queue.

“The badge should know what session I am in, who I am trying to meet, and what I want them to walk away with. Printing it a week early guarantees it knows none of those things.”

Why e-paper, and why it is the only display that makes sense here

It is worth being precise about this, because “because e-ink looks cool” is not an engineering answer — and because the usual complaint about e-paper turns out to be completely irrelevant for a badge.

1. It holds the image with the power off

This is the property everything else hangs off. E-ink pixels are physical pigment particles held in place by nothing but their own inertia. Once the image is written, the panel needs zero power to keep showing it.

An OLED or LCD badge is a battery you wear around your neck, and it dies somewhere around the second keynote. This badge draws itself, opens a setup portal, and then deep-sleeps at microamps with the panel power completely off. You could disconnect the battery entirely and it would still read correctly on day three. The display is not a running process; it is a printed page that happens to be reprintable.

2. It reads like print, because it is reflective

Badges get read at arm’s length, by strangers, in whatever lighting the venue has. Conference halls are either dim and cavernous or flooded with window glare, and the coffee break is usually outside in full sun. A backlit screen in direct sunlight is a mirror. E-ink is reflective — the brighter the room, the better it looks, exactly like paper. It also has no refresh flicker and a viewing angle of essentially 180 degrees, so it stays readable when it is hanging at a tilt on a lanyard.

3. The slow refresh does not matter here — and that is the whole point

E-paper’s famous weakness is that a full refresh takes seconds and flashes while it works. That is fatal for a watch face or a game. For a badge it is a non-issue: the content changes a handful of times a day, deliberately, while you are looking at it.

This is what makes a badge close to the ideal e-paper application outside of e-readers. Every strength (persistence, sunlight readability, paper aesthetics, near-zero idle draw) is exactly what the job needs, and the one real weakness costs nothing.

4. Sixteen grey levels means a real photograph

The ED047TC1 panel is 960×540 at 16 grey levels. Dithered properly, that is enough for a recognisable portrait — not a 1-bit blob, an actual photograph with tonal range. A face on a badge does more work than any amount of styling: it is how people match the name to the person they met yesterday.

5. It does not look like a gadget

This one is not technical but it matters. At a glance the badge looks like an unusually nice printed badge. There is no glow, no bezel-lit rectangle, nothing that reads as “wearable screen”. Then somebody watches the panel flash and redraw itself and the reaction is always the same. A glowing screen would have spent that surprise immediately. This one saves it.

The hardware in one line

LilyGo T5 4.7" ESP32-S3: an ED047TC1 e-paper panel, an ESP32-S3 with WiFi, 8 MB of octal PSRAM (mandatory — the two framebuffers are 259 kB each), 16 MB of flash, a LiPo charger, and a button. Roughly the footprint of a real conference badge, which is the reason for the whole exercise.

Making a badge actually interactive

“Interactive” here does not mean blinking LEDs or a game of Snake. It means the badge responds to the situation you are actually in, in the thirty seconds you have to change it.

Edit it from your phone, on the show floor

The badge brings up a WiFi access point and a captive portal. Join Badge-XXXX from your phone, the setup page opens by itself, you type, you tap **Save & redraw**, and the panel updates in front of you. No laptop, no USB cable, no rebuild, no App Store. If you give it your venue or hotel WiFi it joins that instead and lives at `http://badge.local/`, so you can edit it from anywhere in the building.

A QR code that changes meaning during the day

The QR block is not a static image you uploaded. It is generated on the badge from a mode plus your details, and you can switch the mode in seconds:

- **vCard** — builds a full VCARD 3.0 from your name, title, company, email, phone and website. Someone points a camera at your chest and their phone offers to save the contact. That is the forty-awkward-email-spellings problem solved.

- **URL** — point it at the demo you are showing right now, the slide deck from the talk you just gave, or the repo somebody just asked about.
- **WiFi** — emits a `WIFI:T:WPA;S: . . . ;P: . . . ;` string. If you are running a workshop, your badge hands out the network credentials and you never read an SSID out loud again.
- **Text** — anything else you want to hand over.

The encoder picks the smallest QR version that fits the payload, so a short link gives you big chunky modules that scan from across a table, while a long vCard gives you finer ones. That trade-off is visible and it is yours to make.

The “fun fact” line is the actual feature

Name and company is not enough signal to start a conversation from. A single line — *Ask me about swarm robotics, Looking for a co-founder, Once debugged an I2C bus at 3am with a bent paperclip* — does more for hallway networking than any organised icebreaker. And because it is one field in a web form, you can change it between sessions to match the room you are walking into.

Physical controls and a dark mode for the evening

There is a button (GPIO 21). A short press redraws; holding it for 1.5 seconds puts the badge to sleep immediately. The press that wakes the badge is deliberately ignored until you let go, so a slow release cannot put it straight back to sleep. The *Flip 180°* toggle fixes an upside-down badge whichever way your lanyard decided to hang, and the dark theme inverts the whole panel for the evening reception.



Default, dark theme, no photo, and a deliberately over-long name, title and company. The layout adapts rather than clipping.

Things used in this project

Hardware

Component	Qty	Notes
LilyGo T5 4.7" e-paper, ESP32-S3 version	1	The S3 version specifically. The older ESP32 board uses a different driver branch and pinout. Panel: ED047TC1, 960×540, 16 grey levels.
3.7 V LiPo cell, 1000–2000 mAh, JST-PH 2.0	1	Optional but this is a wearable, so realistically required. The board has the charger and the connector on it.
Lanyard with a clip, plus a 3D-printed or laser-cut frame	1	Any badge holder that clears the USB-C port and the button.
USB-C cable	1	Flashing and charging.

Software and tools

Item	Notes
PlatformIO	The VS Code extension, or <code>pip install platformio</code> . Every library dependency is fetched automatically.
LilyGo-EPD47 (esp32s3 branch)	Panel driver and the bundled Roboto fonts. GPL-3.0.
QRCode by Richard Moore	On-device QR encoding. MIT.
ArduinoJson 7.x	Config serialisation. MIT.
A phone with a browser	That is the entire configuration UI.
Python 3 with pillow, numpy, qrcode	Optional. Runs the host-side layout preview so you can iterate on the design without flashing or burning e-ink refresh cycles.

Step 1: Build and flash

There is no filesystem image to upload and no separate data partition step. The entire 22 kB setup page is compiled into the firmware as a raw string, so flashing is the only thing you do.

terminal

```
git clone <your-repo> && cd E-PAPER_CONFERENCE_BADGE
pio run -t upload
pio device monitor
```

The board definition lives in `boards/T5-ePaper-S3.json`, copied verbatim from the LilyGo repo, and `platformio.ini` points at it with `boards_dir = boards`. That means you do not have to clone the driver

repo by hand or install a board package — the octal-PSRAM memory type and the 16 MB partition table come along with it.

```
platformio.ini (abridged)
[platformio]
boards_dir = boards
default_envs = badge

[env:badge]
platform = espressif32@6.12.0
board = T5-ePaper-S3
framework = arduino
upload_speed = 921600
board_build.filesystem = littlefs

build_flags =
    -DBOARD_HAS_PSRAM
    -DARDUINO_USB_CDC_ON_BOOT=1
    -DLILYGO_T5_EPD47_S3
    ; qrcode.c and font rendering use sizeable VLAs; give the loop task room
    -DARDUINO_LOOP_STACK_SIZE=16384

lib_deps =
    https://github.com/Xinyuan-LilyGO/LilyGo-EPD47.git#esp32s3
    https://github.com/ricmoo/QRCode.git
    bblanchon/ArduinoJson@^7.4.2
```

A healthy cold boot looks like this:

```
serial monitor, 115200 baud
[badge] booting
[badge] battery 4.11 V
[badge] content changed, refreshing the panel
[badge] portal at http://192.168.4.1/ (also http://badge.local/)
[badge] will sleep after 10 idle minutes
```

If you see “FATAL: no PSRAM for the framebuffers”

The build is not configured for octal PSRAM. Check that `board = T5-ePaper-S3` is actually being picked up from `boards/` — the `boards_dir` line in `platformio.ini` is what makes that work. The two 259 kB framebuffers have nowhere else to live.

Step 2: First boot and the setup page

- 1 The badge draws itself with placeholder details, so you immediately see whether the panel is alive.
- 2 It starts an access point called Badge-XXXX (the last two bytes of its MAC), password badge1234.
- 3 Join it from your phone. The captive portal should pop up on its own; if your phone is stubborn, browse to `http://192.168.4.1/`.
- 4 Fill in your details, add a photo, tap **Save & redraw**, and watch the panel flash and settle into your badge.

Under **Network** you can give it your venue or home WiFi, and it will join that on the next boot and be reachable at `http://badge.local/`. If the join fails it falls back to its own access point after about 12 seconds, so there is no way to lock yourself out of your own badge.

Tab	What is on it
Details	Name, pronouns, job title, company, the event name for the top band, the fun fact, an optional footer line, and what the QR should contain
Photo	Pick an image or shoot one, frame it with zoom and pan, adjust brightness and contrast, choose a dithering mode
Look	Show photo, border frame, dark theme, 180° flip
Network	Station credentials, plus the badge's own access point name and password
Status	Battery, free heap and PSRAM, uptime; buttons to redraw or blank the screen

How it works: the four interesting problems

Most of this project is ordinary embedded plumbing. Four parts were genuinely awkward and are worth writing down, because if you build anything on this panel you will hit all four.

Problem 1: the panel is landscape and the badge is not

A conference badge is portrait. The ED047TC1 is 960×540 landscape, and the esp32s3 branch of the LilyGo driver has no rotation support at all — there is no `epd_set_rotation()`, and `EPD_WIDTH` / `EPD_HEIGHT` are compile-time constants.

So `canvas.cpp` owns a 540×960 *logical* portrait canvas and maps every single write on the way out:

```
src/canvas.h and canvas.cpp
static const int CANVAS_W = EPD_HEIGHT;    // 540
static const int CANVAS_H = EPD_WIDTH;     // 960

normal : (x, y) -> (CANVAS_H - 1 - y, x)
flipped: (x, y) -> (y, CANVAS_W - 1 - x)
```

Those two mappings are exactly 180 degrees apart, which is a small piece of luck: the *Flip* toggle is free, and it will always fix an upside-down badge no matter which way the lanyard twists.

And then text, which is worse

The driver's `write_string()` only knows how to lay glyphs out horizontally into a 960-wide buffer. It cannot draw rotated text at all. The fix: render each line into a **scratch framebuffer**, then rotate-blit the inked pixels onto the portrait canvas, treating pure white as transparent so a label can sit on top of a filled band or a rule without punching a white hole through it. Inverted text is the same blit with `255 - v`, which preserves the anti-aliased edges instead of hard-thresholding them.

One performance note that matters: only the strip actually used is cleared between lines. Wiping all 259 kB of scratch PSRAM once per label would completely dominate the render time.

Problem 2: a 12-megapixel photo and no JPEG decoder

The badge never sees a JPEG. It has no decoder and no scaler, and adding one would be a lot of flash for very little benefit. Instead, **your phone does all of it** in a `<canvas>`: cover-fit and crop, greyscale, brightness and contrast, Floyd–Steinberg dithering down to 16 levels, then pack two pixels per byte and POST the raw bitmap. The badge streams those bytes straight to flash.

That is how a camera shot reaches a microcontroller with no image pipeline, and it is also why the preview on your phone is *exact* — the preview is not an approximation of the final pixels, it *is* the final pixels.

The wire format is deliberately trivial, so you can script it:

```
EPB1 bitmap format
"EPB1" | uint16 width LE | uint16 height LE | rows
```

EPB1 bitmap format

Each row is `ceil(width / 2)` bytes.
 The LOW nibble holds the even-x pixel. 0 = black, 15 = white
 -- the same packing `epd_draw_pixel()` uses internally.
 A 320 x 320 photo is 51,208 bytes.

Save your text first, then the photo

The target size is **not** fixed. Because the layout is adaptive, the badge reports the exact square the photo will occupy in `/api/status`, and the browser produces pixels at precisely that size. Rescaling an already-dithered image turns its dither pattern into noise, so the order matters: enter your text, save, then do the photo and it lands 1:1.

Problem 3: long names, and the temptation to shrink text

The easy way to fit variable-length text is to shrink the font until it fits. The easy way is wrong for a badge, because a badge that cannot be read from two metres away has failed at its only job.

So blocks are placed from the outside in, and the photo absorbs whatever is left over:

- 1 The **event band** (86 px) and the **QR block** (150 px) are fixed. A footer line, if you set one, takes its 29 px off the bottom first.
- 2 The **identity block** (name, pronouns, title, company, rule) is *measured* by a dry run of the same drawing code that will later draw it — so the measurement can never drift from what actually appears.
- 3 Each heading takes the largest face that fits on one line. If nothing in the ladder fits, it **wraps at the largest face that fits in two lines** rather than shrinking to something unreadable. A wrapped name reads far better than a tiny one.
- 4 The **fun fact** reserves what it genuinely needs, up to three lines.
- 5 The **photo** gets the remainder, clamped to 180–340 px. If a long name and a long fun fact leave no room, the photo is dropped rather than the text.

The result: a terse badge gets a big portrait, a wordy one gets a smaller portrait, and in both cases everything stays legible. Nothing is ever silently clipped.

src/badge_render.cpp

```
// A centred heading. Prefers the largest face that fits on one line;
// failing that, wraps at the largest face that fits in two -- a wrapped
// heading reads far better than one shrunk to the smallest size.
for (int i = 0; i < nFonts; i++)
    if (canvasTextWidth(fonts[i], s.c_str()) <= contentW) {
        canvasDrawText(CANVAS_W / 2, y, fonts[i], s.c_str(), ALIGN_CENTER, false);
        return fonts[i]->advance_y;
    }

for (int i = 0; i < nFonts; i++) {
    if (longestWordW(fonts[i], s) > contentW) continue;
    int need = canvasDrawParagraph(0, 0, contentW, 8, fonts[i], s.c_str(),
        ALIGN_CENTER, false, /*measureOnly=*/true);
```

```
src/badge_render.cpp
    if (need >= 1 && need <= 2) { ... }
}
```

Problem 4: not wasting refreshes

A full refresh takes several seconds, flashes the panel, and costs an ink cycle. Doing one on every wake would be wasteful and visibly annoying. So the firmware keeps a fingerprint of the config plus the photo's size in **RTC memory**, which survives deep sleep. On wake it compares the fingerprint against what it last drew and, if nothing moved, goes straight to serving the portal without touching the panel at all.

```
src/badge_render.cpp
RTC_DATA_ATTR static uint32_t s_shownFingerprint = 0;
RTC_DATA_ATTR static bool     s_panelValid      = false;

bool badgeNeedsRender()
{
    return !s_panelValid || configFingerprint() != s_shownFingerprint;
}
```

The same discipline applies on the HTTP side. A panel refresh would block the socket for seconds and leave your phone staring at a spinner, so handlers only raise a flag and the refresh runs in `portalLoop()` once the response has been flushed. No request ever waits on the panel.

A bonus problem: José Muñoz became Jos Muoz

Only the mid-size Roboto face bundled with the driver carries Latin-1; Roboto12, 18 and 32 are ASCII only, and the renderer silently skips glyphs it does not have. That quietly turned accented names into mangled ones — a bug that is easy to ship and embarrassing to ship on a name badge. Every string is now folded to ASCII first (accents stripped, smart quotes and dashes normalised) using tables in `src/ascii_fold.h` generated from the Unicode database. It is a compromise, not a fix — see *Limitations*.

Step 3: Power, sleep and the button

Sleep after	Behaviour
10 (default)	Portal stays up for 10 idle minutes, then deep sleep. The image stays.
0	WiFi stays on indefinitely. Fine on a desk, hungry on a battery.
1–240	Anything in between.

The button (GPIO 21): short press redraws, hold for 1.5 seconds to sleep immediately. The press that woke the badge is ignored until you release it, so a slow thumb cannot wake the badge and put it straight back to sleep.

Battery sensing has one real gotcha. The board divides VBAT by two into GPIO 14, which is on **ADC2** — and the WiFi radio takes ADC2 over completely once it starts. So the voltage is sampled exactly once, at boot, before `portalBegin()` is called. If you want live readings you would have to stop the radio first, which is not worth it here.

```
src/main.cpp
// ADC2 is unusable once WiFi is running, so this has to happen
// before portalBegin().
static void readBattery()
{
    analogSetPinAttenuation(BATT_PIN, ADC_11db);
    uint32_t acc = 0;
    for (int i = 0; i < 16; i++) acc += analogReadMilliVolts(BATT_PIN);
    s_batt = (acc / 16.0f) * 2.0f / 1000.0f;
}
```

Current build for reference: **965 kB of flash** (15% of the app partition), **48 kB of internal RAM** (15%), plus 506 kB of PSRAM for the two framebuffers. There is a great deal of room left to build on.

Step 4: Making it yours — every layer is customisable

This is the part I care most about. There are four levels of customisation, and you can stop at whichever one you like.

Level 1 — the web form (no code, no cable)

Twenty-two settings, all editable from a phone. Every visible string, what the QR means, four appearance toggles, the sleep timeout, and both sets of WiFi credentials. Stored as `/badge.json` on LittleFS, written to a temp file and renamed, so a power cut mid-save cannot leave a truncated config behind.

Key	Type	Default	Notes
name	string	Your Name	Wraps to two lines if long
pronouns	string	""	Small line under the name
profession	string	Your Job Title	
company	string	Your Company	Rendered in caps
funFact	string	(placeholder)	About 130 chars fit at the larger size
event	string	CONFERENCE 2026	Top band, rendered in caps
footer	string	""	Optional line at the very bottom
qrMode	enum	url	url / text / vcard / wifi
qrText	string	https://example.com	Payload; for wifi use SSID password
qrCaption	string	Scan to connect	Beside the QR
email / phone	string	""	Go into the vCard
website	string	""	Printed under the caption
showPhoto	bool	true	Off gives the text more room
flip	bool	false	Rotate the layout 180 degrees
invert	bool	false	Dark theme
border	bool	true	Hairline frame around the badge
sleepAfterMin	int	10	0 = never sleep; capped at 240
apSsid / apPass	string	"" / badge1234	Empty SSID means Badge-XXXX from the MAC
staSsid / staPass	string	""	Leave empty to stay in AP mode

Level 2 — the layout constants

The proportions of the badge are ten `#defines` at the top of `src/badge_render.cpp`. Want a bigger portrait and a tighter fun fact? Change two numbers.

```
src/badge_render.cpp
#define M          22    // outer margin
#define HDR_H      86    // event band height
#define QR_BOX     150   // target QR side, quantised down to whole modules
#define RULE       3
#define PHOTO_GAP  26    // breathing room under the photo
#define FACT_GAP   18    // breathing room above the QR block
#define FACT_PAD   12    // inside the fun fact box
#define FACT_LINES 3     // most lines reserved for the fun fact up front
#define PHOTO_MIN 180
#define PHOTO_MAX 340
```

Iterate without flashing: the host-side preview

`tools/preview.py` re-implements the glyph renderer, the portrait canvas and the layout in Python, reading the **real compressed font data** straight out of the library headers — so the text metrics are exact, not estimated. It renders the badge to a PNG. Change a constant, run it, look at the picture, repeat. No flashing, no e-ink cycles, no waiting.

```
terminal
pip install pillow numpy qrcode
pio run                                # once, so the fonts land in .pio/
python tools/preview.py --out tools/preview [--dark] [--no-photo] [--flip]
python tools/gallery.py                # regenerate the documentation images
```

It writes `_badge.png` (what a person sees) and `_panel.png` (the raw framebuffer), and prints the decisions it made:

```
preview output
- photo 218px, identity 233px, slack 386px, QR top y=749
- Ryan Kiprotich: Roboto32 x1
- Embedded Systems E: Roboto18 x1
- fun fact 142px box, Roboto12, 3 line(s)
```

This tool caught two real layout bugs during development — a fun fact that was being silently dropped, and headings that shrank instead of wrapping. It is a reimplementation, not the firmware, so if you move things around in `badge_render.cpp`, keep it in step or it will lie to you.

Level 3 — new fields, new fonts, a different badge entirely

Adding a field

Four places, and the fourth is the one people forget:

- 1 The struct in `badge_config.h`

- 2 applyJson and fillDoc in badge_config.cpp
- 3 The form in web_ui.h (add the id to TEXT_KEYS)
- 4 The layout in badge_render.cpp — **and** configFingerprint(), or changing your new field will not trigger a redraw and you will spend an hour convinced the save is broken.

Adding a font

Roboto12 / 18 / Roboto / Roboto32 ship with the driver. For another size, or a face with wider Unicode coverage, use the EPD47 font converter from the LilyGo repo, drop the generated header in src/, include it from badge_render.cpp *only* (they are const at file scope, so including them twice just duplicates the data in flash), and add it to the font ladders:

src/badge_render.cpp

```
static const GFXfont *NAME_FONTS[] = { &Roboto32, &Roboto, &Roboto18 };
static const GFXfont *TITLE_FONTS[] = { &Roboto, &Roboto18, &Roboto12 };
static const GFXfont *SMALL_FONTS[] = { &Roboto18, &Roboto12 };
```

A ladder is just an ordered preference list: largest first, and the layout walks down it until something fits. Adding a face is a one-line change.

Level 4 — the HTTP API, and badging a whole team

Everything the setup page can do is a plain HTTP call, which means anything you can script can drive the badge.

Method	Path	Notes
GET	/	The setup page
GET	/api/config	Settings as JSON; passwords masked as __unchanged__
POST	/api/config[?render=1]	JSON body. Send the whole object — missing keys reset to defaults
GET	/api/status	Mode, IP, RSSI, battery, heap, PSRAM, uptime, and the live photo box size
POST	/api/render	Redraw with two ghost-clearing passes
POST	/api/blank	Wipe to white
GET	/api/photo	The stored EPB1 bitmap
POST	/api/photo[?render=1]	Multipart upload, field name photo
DELETE	/api/photo	Remove the stored photo

terminal

```
# read the current settings
curl http://badge.local/api/config

# change things (fetch, edit, post the whole object back)
curl -X POST 'http://badge.local/api/config?render=1' \
```

terminal

```
-H 'Content-Type: application/json' \
-d '{"name":"Ryan Kiprotich","company":"Waziup", ... }'

# upload a photo you packed yourself
curl -X POST 'http://badge.local/api/photo?render=1' -F photo=@photo.bin
```

Uploads are streamed straight to a temp file and only promoted over `/photo.bin` once the EPB1 header and length check pass, so a dropped connection never corrupts the photo you already had.

Idea: badge an entire team from a spreadsheet

Flash N boards, give them all the same station credentials, and each one comes up on the network. Walk a CSV of names and titles, GET `/api/config`, patch the fields, POST it back with `?render=1`, then push each portrait to `/api/photo`. Remember that POST `/api/config` is **not** a patch — fetch, edit, and send the whole object, or every field you omitted will snap back to its default.

Troubleshooting

Symptom	What is going on
The badge is upside down.	Tick <i>Flip 180°</i> under Look. The two rotation mappings are exactly 180° apart, so this always works.
FATAL: no PSRAM for the framebuffers	The build is not configured for octal PSRAM. Confirm board = T5-ePaper-S3 is being read from boards/.
Battery always reads 0 V.	It is sampled once at boot, before WiFi, because GPIO 14 is on ADC2. Live readings would require stopping the radio.
Ghosting.	Every content refresh already runs two clear cycles. For a stubborn shadow, hit <i>Clear screen to white</i> on the Status tab a couple of times. Leaving the panel white before storing it for months is kind to it.
The QR will not scan.	Shorten the payload. The encoder picks the smallest version that fits, so a short link gives chunky modules while a full vCard gives small ones. <code>https://</code> links scan well at this size; long vCards are marginal. The serial log warns if the payload will not fit at all.
A character vanished or became “?”.	Anything outside Latin-1 and Latin Extended-A folds to ?. See the ASCII folding note above.
The page loads but saving fails.	Check free space on LittleFS via the Status tab. A failed photo upload leaves nothing behind, but a full filesystem will reject badge.json writes.
It went to sleep while I was typing.	Only HTTP requests count as activity. The page polls <code>/api/status</code> every 20 seconds to keep the timer at bay, but a backgrounded phone browser gets throttled. Set <code>sleepAfterMin</code> to 0 while you are setting things up.

Honest limitations

- **Latin script only.** The bundled fonts do not carry CJK, Arabic, Devanagari or Cyrillic, and everything is folded to ASCII before rendering. For an international conference this is a real shortcoming, and the fix is a font with wider coverage converted through the EPD47 tool — the code path is already there, it is the glyph data that is missing.
- **Partial refresh is not used.** Every update is a full-panel refresh. Slower, but it avoids accumulating ghosts on a badge that changes rarely.
- **One client at a time.** The synchronous WebServer is a deliberate choice — stable, dependency-free, and a badge has exactly one user.
- **The AP password is stored in plain text** in `badge.json`, like every other Arduino WiFi sketch. Anyone who can read the flash can read it.
- `POST /api/config` **is not a patch.** Missing keys reset to defaults. Fetch, edit, post the whole object.
- `tools/preview.py` **is a reimplementation**, not the firmware. It can drift out of step if you change the layout and forget it.

Where to take it next

- **Session-aware badges.** Pull the conference schedule over WiFi and rotate the fun fact and QR to match whatever talk is happening right now.
- **Badge-to-badge.** ESP-NOW between badges: exchange vCards by bumping shoulders, and keep a list of who you met.
- **Organiser mode.** An MQTT topic per badge so the event can push a track colour, a room change, or an evacuation notice to 200 badges at once.
- **Partial refresh for a clock or a queue number.** The panel supports it; this firmware deliberately does not use it, but a small partial-update region at the bottom would be a natural extension.
- **An e-ink desk sign.** The same firmware, a stand instead of a lanyard, and an “in a meeting until” line. It already sleeps for weeks on a charge.

Credits and licensing

Component	Licence
LilyGo-EPD47 (esp32s3 branch)	GPL-3.0
QRCode by Richard Moore	MIT
ArduinoJson by Benoit Blanchon	MIT
Roboto typeface	Apache-2.0

Note the driver's licence before you redistribute

This firmware links against LilyGo-EPD47, which is GPL-3.0. If you distribute binaries or publish a fork, the combined work is effectively GPL-3.0 and you need to offer the corresponding source. Keeping it to yourself imposes no obligation. If you need a permissive licence, the driver is the piece that would have to be replaced — the badge code itself has no other copyleft dependency.

Source layout at a glance.

File	Responsibility
src/main.cpp	Boot, battery, button, deep sleep
src/canvas.{h,cpp}	Portrait canvas over the landscape panel: pixels, rects, text, QR, photo, ASCII folding
src/badge_render.{h,cpp}	The badge layout and its measure/draw passes
src/badge_config.{h,cpp}	Settings, LittleFS persistence, QR payload building, fingerprinting
src/portal.{h,cpp}	WiFi, captive portal, DNS, HTTP API
src/web_ui.h	The whole setup page, embedded in flash as a raw string
src/ascii_fold.h	Generated Unicode-to-ASCII tables
boards/T5-ePaper-S3.json	Board definition, copied from the LilyGo repo
tools/preview.py	Host-side layout preview
tools/gallery.py	Regenerates the documentation images